



Blue



a parser generator



What is Blue?

- Parser generator
- Given BNF-style grammar, generates Racket structs and methods to parse text into those structs
- Supports custom tokens defined via Regexp

```
#lang racket
(require "../blue.rkt")
(generate lang "../binding.blue")

(lang-binding-parse "let x = 2;")
```

Racket generate creates structs and methods to parse into them

Principles

Tokens

- Basic building blocks
- Defined via regular expressions
- Priority comes from definition order
- Supports all Racket regexp features

Examples:

```
ZERO  = "0"  
ONE   = "1"  
IDENT = "[a-zA-Z_]+"  
LET   = "\let"  
EQ     = "="
```

Nodes

- Actual output
- Bind other nodes or tokens to fields
- May be recursive
- Created as Racket structs

Examples:

```
BinDigit ::= ZERO as val | ONE as val  
BinNum   ::= BinDigit as fst BinNum as rst  
           | BinDigit as fst  
Binding  ::= LET IDENT as name EQ prim as val
```

Formal Grammar

```
Identifier    = [a-zA-Z\-\_][a-zA-Z0-9\-\_]+
String        = " .+ "
FieldRHS      ::= Identifier
               | (Identifier "as" Identifier)
               | (FieldRHS "|" FieldRHS)
Field         ::= Identifier "[:]=" FieldRHS <EOL>
TokenField    ::= Identifier "=" String <EOL>
Grammar       ::= (Field | TokenField)+
```

Implementation – Lexing/Parsing

- Lexes into tokens (strings, identifiers) and operators (=, ::, |, as)
- Parses into Field and TokenField structs
- Structured AST output, defined by the grammar
- Parsed data is consumed by the generator

Implementation – Generation

For each Blue language we generate:

- A tokenizer
- Struct definitions
- Methods to parse from both tokens and strings
- Macro expansion to be explained soon

Examples...